



Mesurer pour comprendre

From *'it worked on my notebook'* to production-ready machine learning

Bridging the gap with MLOps

2026-06-04

Lino GALIANA

lino.galiana@insee.fr

Insee — French National Institute of Statistics and Economic Studies

Insee

Sommaire



01

Introduction



02

**Handling data driven
projects distinct lifecycles**



03

**From zero to hero in
production**



04

**Adding experiment tracking
and observability**



05

Conclusion



01

Introduction

The “production wall”

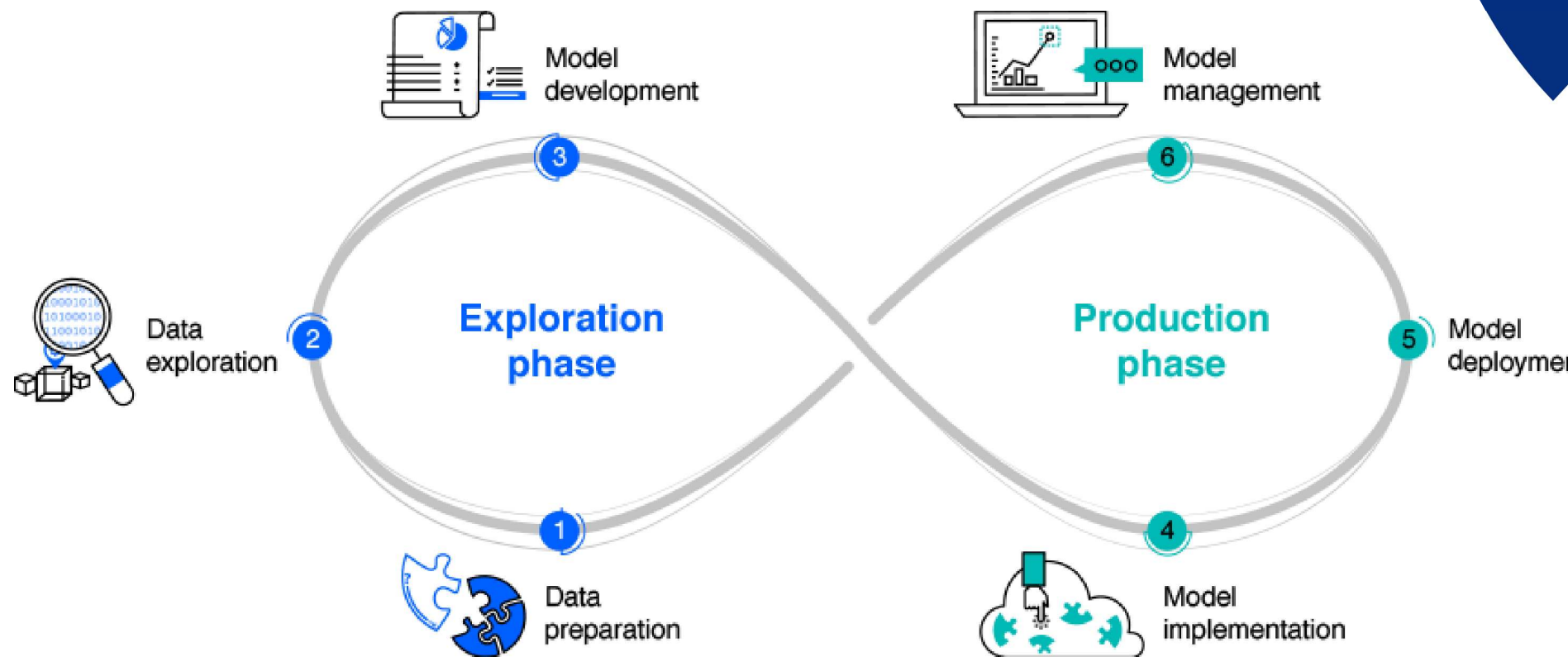
- Most data-driven projects **never deliver value** (1, 2, 3)
 - Most *data science* and AI POCs fail when going to production
 - How to **move beyond the experimentation stage**?

Defining production

Going to production: making an application **live** in the space of its **users**

- **Serving**: **deploying** the application in a **relevant format** for its potential users
 - **Keeping it alive**: managing the **lifecycle** and fostering **continuous improvement**
-
- **Multiple dimensions**: domain knowledge, organisation, infrastructure, technical to

Fostering continuity



Source: [ibm.com](https://www.ibm.com)

– Applying and extending **software development best practices**

- **DataOps**: building robust **data pipelines**
- **MLOps**: deploying and maintaining **ML models**

Insee

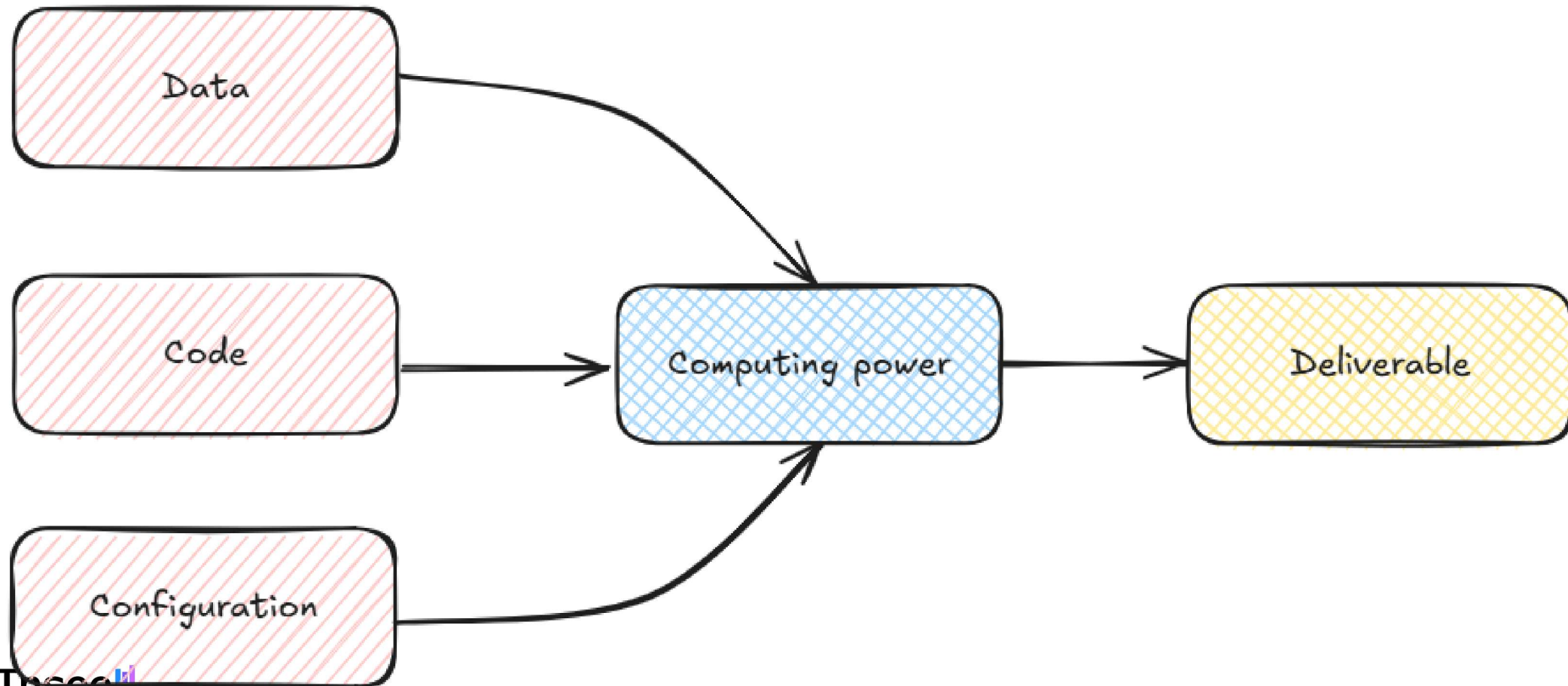
The background features several large, overlapping geometric shapes in various shades of purple and pink. These shapes include triangles, quadrilaterals, and polygons, some of which are partially cut off by the edges of the frame. The colors range from a deep, dark purple to a light, pale pink.

02

Handling data driven projects distinct lifecycles

Three lifecycles to master

- A robust ML pipeline requires managing **three independent lifecycles**:
 - *Jupyter Notebooks* does not separate them properly



Why does this matter?

- Models (weights and inference) are **not static artefacts**:
 - they depend on data, code, *and* the environment in which they were trained
- A model result is **only reproducible** if all three are tracked together
- Failing to manage even one lifecycle leads to:
 - “*Works on my machine*” syndrome
 - Inability to roll back a broken update
 - Silent regressions caused by data or dependency drift

Choosing appropriate tools

Code

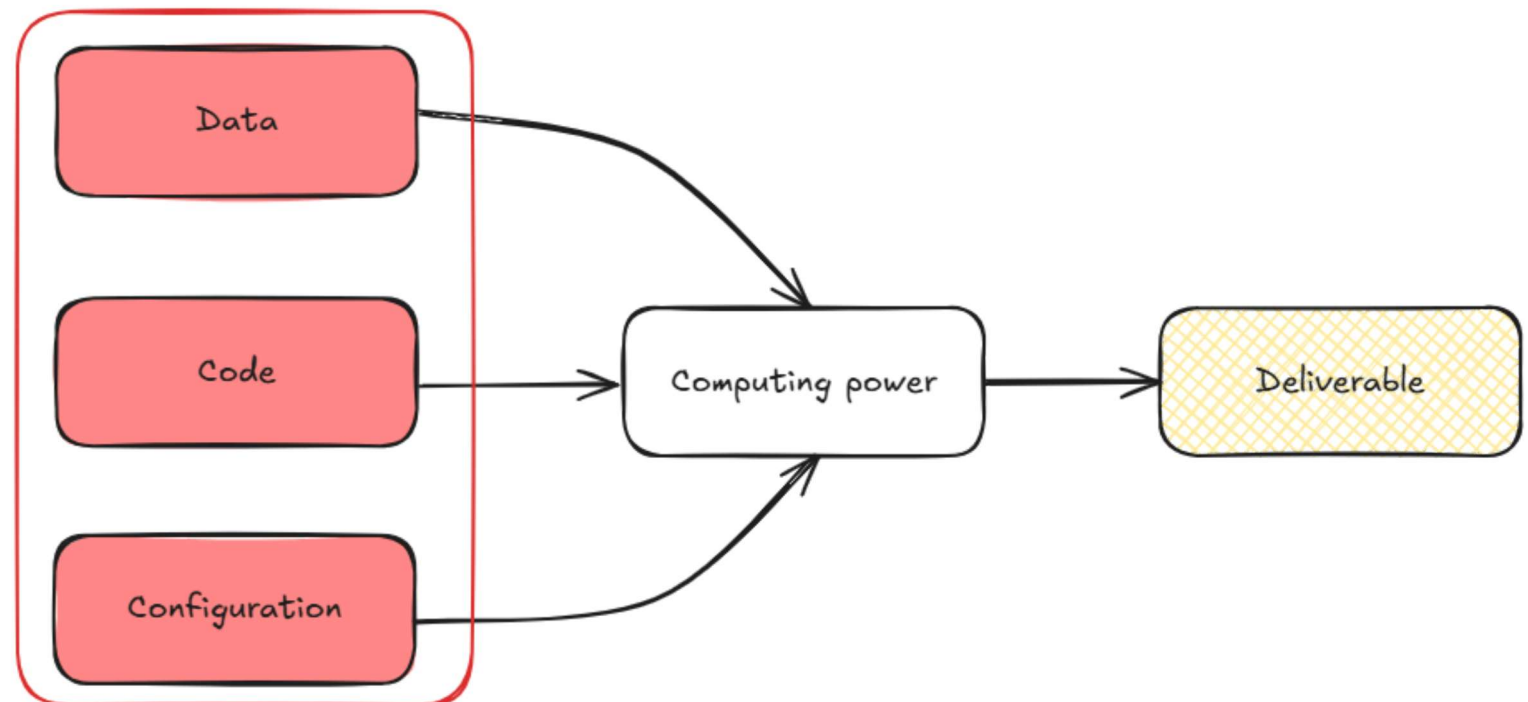
Versioning ([Git](#)), improving quality with formatters ([Ruff](#)), community standard structure ([cookiecutters](#))...

Configuration

Virtual environments and dependency management ([uv](#)), controlling external dependencies ([Docker](#))...

Data

Standardised cloud storage, oriented work...





03

From zero to hero in produ

“It works on my machine”

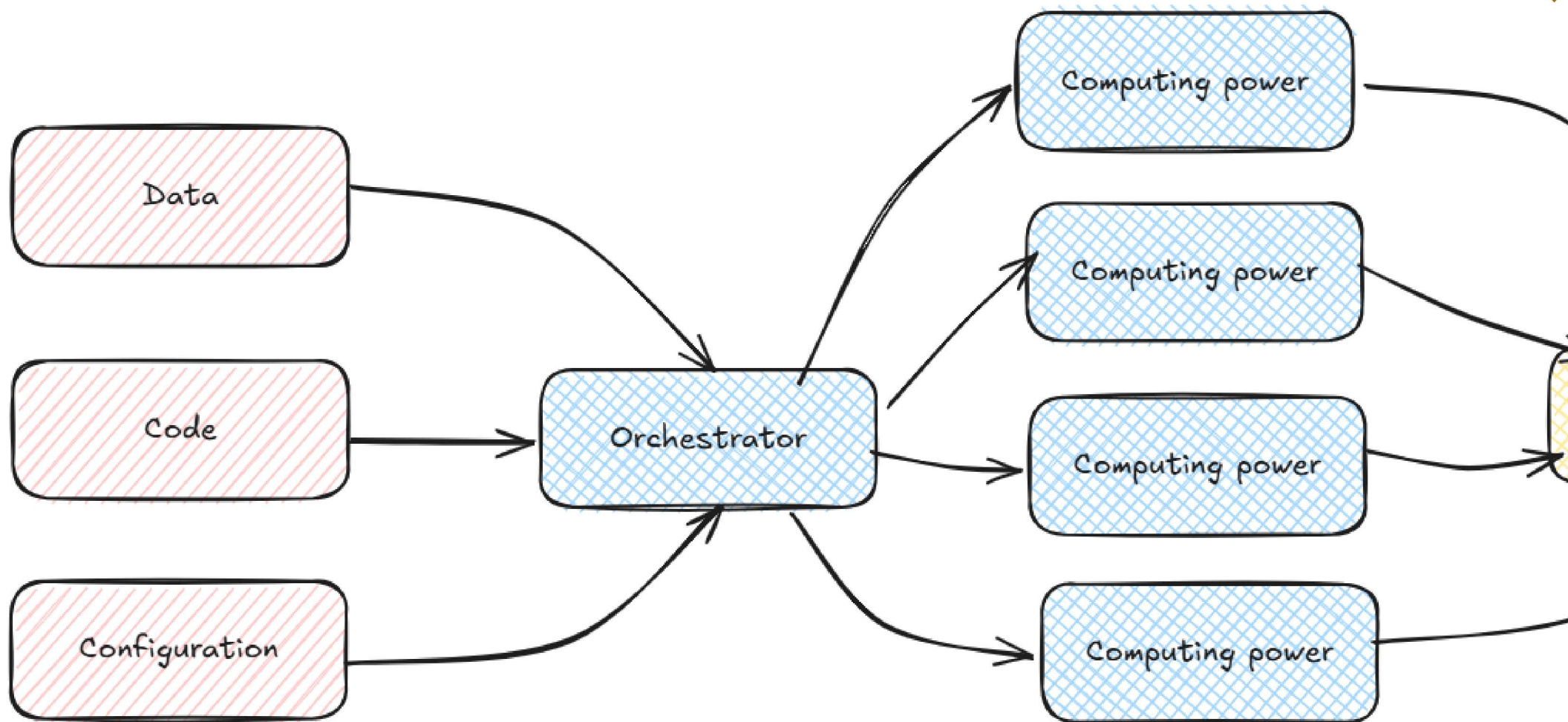


Bridging the dev/production gap

- **Environment gap** is one of the most common sources of failure:
 - Production servers might use different OS, libraries, CUDA versions...
- **Containerisation** ([Docker](#)) solves this by packaging the full runtime alongside the code
 - Same image runs locally, in CI, and in production
 - Eliminates *“it works on my machine”* class of bugs

An industrialized project

Kubernetes turns individual containers into an **industrialised, scalable fleet**



MLOps on Kubernetes

- **Training**: launch parallel trainings (e.g. cross validation)
- **Model serving**: expose a versioned model behind a stable endpoint (API)
- **Canary deployments**: route a fraction of traffic to a new model version before full
- **Rollbacks**: switch back to a previous version if performance degrades

Note

Great improvement but this is only the first phase of a project: continuous improvement requires observability



04

Adding experiment tracking and observability

Experimentation phase

During development, practitioners need to:

- Track **every experiment**: hyperparameters, metrics, artefacts
- **Compare runs** objectively
- **Select and register** the best model version

MLFlow (and similar platforms!) provides a centralised tracking server, model registry, a

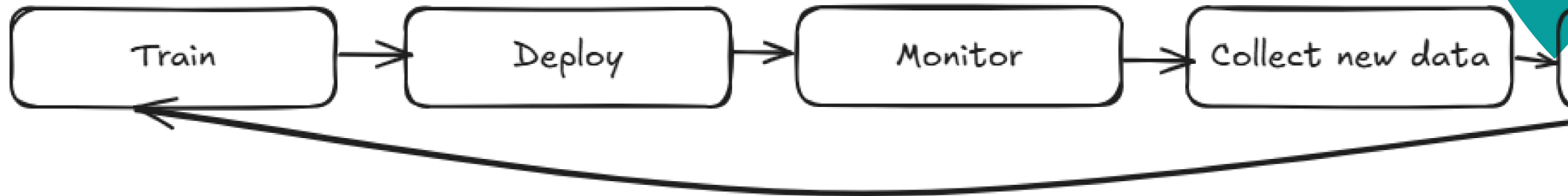
From experimentation to production observability

- The same platforms extend into **production monitoring**:
 - Log **real-world inputs and outputs**
 - Compute **performance metrics** against ground truth or human feedback
 - Detect **data drift** and **concept drift**

LLM-based systems need additional tracking. [Langfuse](#) adds:

- Trace-level observability (prompt → retrieval → generation)
- Cost and latency tracking
- Human annotation workflows

Feedback loops and continuous improvement



- **Monitoring is not optional**: a model that worked at launch will degrade as the world changes
- **Feedback loops** close the gap between offline evaluation and real-world performance
- Good observability turns every production incident into a **training signal**

Two paradigms, two sets of operational constraints

	Supervised ML	LLM-based systems
Training	Full retraining cycle	Fine-tuning or prompt engineering
Evaluation	Standard metrics (F1, RMSE...)	Requires LLM-as-judge or human
Drift	Feature / label drift	Prompt drift, outdated knowledge
Availability	Batch or on the fly ?	Continuous
Infrastructure	CPU often sufficient for inference	Bigger and bigger GPU (💰 💰)

Annotation and evaluation challenges

Supervised learning:

- Ground truth is (relatively) well-defined
- Evaluation is largely automated

LLMs:

- What is the “correct” output? Often ambiguous
- Human evaluation is expensive and hard to scale
- Automated evaluation (LLM-as-judge) introduces its own biases
- Prompt changes can silently break previously passing evaluations

Avertissement

Is it really possible to leapfrog when having missed the ML era ?



05

Conclusion

Key takeaways

1. **Structure your project** around three independent lifecycles: data, code, environment
2. **Track everything**: experiments, models, prompts — if it's not tracked, it didn't happen
3. **Monitor in production**: evaluation does not stop at deployment
4. **Know your paradigm**: supervised ML and LLM-based systems require different tooling and processes

The gap between a notebook that works and a system that delivers value is not a technical gap: it is an operational one.